

Apostila Técnica — Site Pontos de Fuga

Como o sistema funciona: um guia para entender o código

***Para quem é este documento?** Para qualquer pessoa que queira entender como este site foi construído, mesmo sem experiência prévia em programação. Os conceitos são introduzidos do zero, com analogias do mundo real, antes de mostrar o código.*

Sumário

- 1. O que é este projeto e como está organizado
- 2. Conceitos fundamentais de programação (leia antes de tudo)
- 3. O banco de dados do site — `data.json`
- 4. O carregador de dados — `data.js`
- 5. O código compartilhado — `script-shared.js`
- 6. A página inicial — `script-index.js`
- 7. A página de produções — `script-producoes.js`
- 8. A página de cada filme — `script-filme.js`
- 9. A página nossa história — `script-historia.js`
- 10. A página vem aí — `script-vemai.js`
- 11. A página de contato — `script-contato.js`
- 12. Glossário de termos técnicos

Capítulo 1 — O que é este projeto e como está organizado

O que é o site

O site da **Pontos de Fuga** é um catálogo digital de filmes de uma produtora cinematográfica. Ele exibe os filmes produzidos, os projetos em andamento, a história da empresa e um formulário de contato.

Como um site funciona (visão geral)

Um site é feito de três tipos de arquivo que trabalham juntos:

Tipo	Arquivo	Responsabilidade
HTML	<code>.html</code>	A estrutura — o esqueleto da página (onde ficam o menu, os títulos, as seções)
CSS	<code>.css</code>	A aparência — as cores, fontes, tamanhos, animações
JavaScript	<code>.js</code>	O comportamento — o que acontece quando você clica, o que é carregado dinamicamente

Analogia: pense num carro. O HTML é a carroceria e os bancos (estrutura). O CSS é a pintura, o estofado, o design (aparência). O JavaScript é o motor e a eletrônica (comportamento).

Estrutura de pastas do projeto

```
site_produtores/
|
|— index.html           ← Página inicial
|— producoes.html       ← Lista de todos os filmes
|— filme.html           ← Página de detalhe de um filme
|— historia.html        ← Nossa história
|— vemai.html           ← Filmes em produção
|— contato.html         ← Formulário de contato
|— admin.html           ← Painel de administração (protegido por senha)
|
|— scripts/
|   |— data.json         ← O banco de dados (todos os textos e dados)
|   |— data.js           ← Carrega o data.json e disponibiliza os dados
|   |— script-shared.js  ← Código que funciona em TODAS as páginas
|   |— script-index.js   ← Código exclusivo da página inicial
|   |— script-producoes.js
|   |— script-filme.js
|   |— script-historia.js
|   |— script-vemai.js
|   |— script-contato.js
|   |— script-admin.js
|
|— style/
|   |— style-base.css    ← Estilos que valem para todas as páginas
|   |— style-index.css
|   |— ... (um CSS por página)
|
|— assets/
|   |— ... (imagens, vídeos, logos)
```

Por que não tem um servidor?

A maioria dos sites modernos tem um servidor — um computador remoto que processa pedidos e gera páginas. Este site é diferente: é um **site estático**. Não há processamento no servidor. Todos os arquivos são enviados direto para o navegador do visitante, que executa o JavaScript localmente.

Os dados são armazenados no arquivo `data.json`, dentro do próprio repositório no GitHub. Quando o administrador precisa atualizar um filme, ele usa o painel admin, que salva as alterações diretamente nesse arquivo via API do GitHub.

Capítulo 2 — Conceitos Fundamentais de Programação

Antes de ler o código, você precisa entender os blocos de construção da linguagem JavaScript. Este capítulo apresenta cada conceito com uma analogia, depois mostra como ele aparece no código.

2.1 — Variáveis

O que é: uma caixa com nome que guarda um valor. Você pode colocar qualquer coisa dentro: um número, um texto, uma lista.

Analogia: uma variável é como uma etiqueta num pote. O nome da etiqueta é o nome da variável. O que está dentro do pote é o valor.

```
// Declarando variáveis
let filmeAtual = 'Simonal';      // texto (chamado de "string")
let anoLancamento = 2023;      // número
let estaEmCartaz = true;        // verdadeiro/falso (chamado de "boolean")
```

Diferença entre `let` e `const` :

- `let` — o valor pode mudar depois
- `const` — o valor nunca muda (é uma constante)

```
let slideAtual = 0;              // vai mudar conforme o carrossel avança
const PALETA = ['#c9a84c', '#c4622d']; // a paleta nunca muda
```

2.2 — Funções

O que é: um bloco de código com nome que executa uma tarefa. Você define uma vez, usa quantas vezes quiser.

Analogia: uma função é como uma receita de bolo. Você escreve a receita uma vez. Toda vez que quiser fazer um bolo, é só seguir a receita. Você pode fazer o bolo para pessoas diferentes (parâmetros diferentes), mas a receita é a mesma.

```
// Definindo uma função
function saudar(nome) {
  return 'Olá, ' + nome + '!';
}

// Usando (chamando) a função
saudar('Pedro'); // retorna 'Olá, Pedro!'
saudar('Maria'); // retorna 'Olá, Maria!'
```

Parâmetros são os ingredientes da receita — valores que a função recebe para trabalhar. `return` é o resultado que a função devolve.

No site, as funções de renderização são como receitas para criar HTML:

```
function criarCard(filme) {
  return '<div class="card">' + filme.title + '</div>';
}
```

2.3 — Arrays (Listas)

O que é: uma lista ordenada de itens, identificados por sua posição (começando em 0).

Analogia: uma estante de livros numerada. O livro na posição 0 é o primeiro, na posição 1 é o segundo, etc.

```
const filmes = ['Simonal', 'O Amor Dá Voltas', 'Design Brasileiro'];

filmes[0] // 'Simonal' (primeiro item)
filmes[1] // 'O Amor Dá Voltas' (segundo item)
filmes.length // 3 (quantidade de itens)
```

Métodos importantes de arrays (operações que você pode fazer):

- `.map()` — transforma cada item em outra coisa, gerando uma nova lista
- `.filter()` — cria uma nova lista só com os itens que passam por uma condição
- `.forEach()` — percorre cada item e faz algo com ele (sem criar nova lista)
- `.sort()` — ordena a lista
- `.join()` — junta todos os itens em uma única string

```
const anos = filmes.map(f => f.year);
// Pega o ano de cada filme e cria uma nova lista só com os anos
```

2.4 — Objetos

O que é: uma coleção de propriedades nomeadas. Diferente do array (que usa número para acessar), o objeto usa nome.

Analogia: uma ficha de cadastro. Uma ficha tem campos com nome: "Nome", "Endereço", "Telefone". Um objeto é exatamente isso — campos com nome e valor.

```
const filme = {
  title: 'Simonal',
  director: 'Cláudio Manoel',
  year: 2024,
  hero: true
};

// Acessando as propriedades
filme.title // 'Simonal'
filme.director // 'Cláudio Manoel'
filme.year // 2024
```

No `data.json`, cada filme é um objeto. A lista de filmes é um array de objetos.

2.5 — Condicionais (if/else)

O que é: uma bifurcação. "Se isso for verdade, faça X. Senão, faça Y."

Analogia: uma encruzilhada. Dependendo da condição, você vai por um caminho ou pelo outro.

```
if (idioma === 'en') {
  titulo = filme.titleEn; // título em inglês
} else {
  titulo = filme.title;    // título em português
}
```

Forma compacta (operador ternário): para casos simples, existe uma versão de uma linha:

```
// condição ? valor_se_verdadeiro : valor_se_falso
const titulo = idioma === 'en' ? filme.titleEn : filme.title;
```

Leia como: "se idioma é 'en', use titleEn; caso contrário, use title."

Operador `||` (ou) como fallback:

```
const texto = pagesData.index.ctaTitlePt || 'Tem uma história?';
```

Leia como: "use o texto do admin, mas se estiver vazio, use 'Tem uma história?'". Se o primeiro valor for vazio/nulo, o `||` usa o segundo.

2.6 — O DOM — Como o JavaScript toca no HTML

O que é o DOM: quando o navegador lê um arquivo HTML, ele cria uma representação em memória de todos os elementos da página. Essa representação se chama **DOM** (Document Object Model). O JavaScript consegue ler e modificar o DOM — é assim que ele altera o que você vê na tela.

Analogia: o DOM é como uma planta baixa de um prédio. Cada cômodo é um elemento HTML. O JavaScript é o arquiteto que pode consultar a planta e fazer reformas em tempo real.

```
// Encontrando elementos
document.getElementById('meuTitulo') // pelo ID (único na página)
document.querySelector('.meu-card')  // pelo seletor CSS (pega o primeiro)
document.querySelectorAll('.meu-card') // pega TODOS os elementos

// Modificando elementos
elemento.textContent = 'Novo texto'; // muda o texto
elemento.innerHTML = '<b>Texto em negrito</b>'; // muda o HTML interno
elemento.style.color = 'red';         // muda o estilo
elemento.classList.add('ativo');      // adiciona uma classe CSS
elemento.classList.remove('ativo');   // remove uma classe CSS
```

2.7 — Eventos

O que é: o navegador "dispara" um evento quando algo acontece — um clique, um movimento do mouse, o scroll da página. Você pode registrar funções para serem chamadas quando esses eventos acontecem.

Analogia: um alarme. Você configura: "quando o telefone tocar, atenda." No código: "quando esse botão for clicado, execute essa função."

```
botao.addEventListener('click', function() {  
  // isso é executado quando o botão for clicado  
  alert('Clicou!');  
});
```

Os eventos mais usados no site:

- 'click' — quando clica
- 'mouseenter' — quando o mouse entra sobre o elemento
- 'mouseleave' — quando o mouse sai do elemento
- 'scroll' — quando a página é rolada
- 'submit' — quando um formulário é enviado
- 'keydown' — quando uma tecla é pressionada

2.8 — Promises e Código Assíncrono

Este é o conceito mais importante para entender o projeto. Leia com atenção.

O problema: algumas operações demoram — buscar um arquivo da internet, por exemplo. O JavaScript não pode simplesmente parar e esperar. Enquanto espera, a página ficaria travada.

A solução: o JavaScript dispara a operação e continua rodando. Quando a operação terminar, uma função de "callback" é chamada com o resultado.

Analogia: você vai a um restaurante e faz o pedido. O garçom não fica parado esperando a comida ficar pronta — ele vai atender outras mesas. Quando a comida ficar pronta, ele te avisa. Você não ficou parado esperando, mas também não comeu antes da hora.

Uma **Promise** é exatamente o comprovante do pedido — ela representa um valor que ainda não chegou, mas vai chegar. Ela pode terminar de duas formas: `fulfilled` (deu certo) ou `rejected` (deu errado).

```
fetch('scripts/data.json')           // "faça o pedido"  
  .then(resposta => {                 // "quando chegar (deu certo)..."  
    return resposta.json();  
  })  
  .then(dados => {                    // "quando converter (deu certo)..."  
    console.log(dados);  
  })  
  .catch(erro => {                    // "se qualquer coisa der errado..."  
    console.error(erro);  
  });
```

Cada `.then()` recebe o resultado do anterior e pode retornar um novo valor (ou uma nova Promise) para o próximo `.then()`.

2.9 — Template Literals (textos com variáveis dentro)

O que é: uma forma de criar strings com valores de variáveis embutidos, usando crases (```) em vez de aspas.

```
const nome = 'Pedro';
const ano = 2024;

// Forma antiga (complicada)
const texto1 = 'Olá, ' + nome + '! O ano é ' + ano + '.';

// Template literal (moderno e legível)
const texto2 = `Olá, ${nome}! O ano é ${ano}.`;
// resultado: 'Olá, Pedro! O ano é 2024.'
```

No site, template literals são usados para criar HTML dinamicamente:

```
const card = `
  <div class="card">
    <h2>${filme.title}</h2>
    <p>Dir. ${filme.director} · ${filme.year}</p>
  </div>
`;
```

2.10 — Spread Operator (...)

O que é: um operador que "espalha" os itens de um array ou as propriedades de um objeto.

Analogia: imagine que você tem uma sacola de compras (array) e quer colocar tudo em outra sacola maior, junto com itens novos. O spread esvazia a primeira sacola dentro da segunda.

```
// Em arrays – cria uma cópia
const original = [1, 2, 3];
const copia = [...original];           // [1, 2, 3] – cópia independente
const ampliado = [...original, 4, 5]; // [1, 2, 3, 4, 5]

// Em objetos – mescla propriedades
const base = { cor: 'azul', tamanho: 'grande' };
const extra = { cor: 'vermelho', peso: '1kg' };
const mesclado = { ...base, ...extra };
// { cor: 'vermelho', tamanho: 'grande', peso: '1kg' }
// 'cor' foi sobrescrita pelo segundo objeto
```

Capítulo 3 — O Banco de Dados: data.json

O que é JSON

JSON (JavaScript Object Notation) é um formato de texto para guardar dados estruturados. É basicamente uma forma de escrever objetos e arrays do JavaScript em um arquivo de texto.

```
{
  "nome": "Simonal",
```

```
"ano": 2024,
"ativo": true,
"generos": ["documentário", "musical"]
}
```

Regras do JSON:

- Chaves (nomes das propriedades) sempre entre aspas duplas
- Valores texto sempre entre aspas duplas
- Números sem aspas
- Listas entre []
- Objetos entre { }

O que tem no data.json

O data.json é o coração do site. Tudo que aparece na tela veio desse arquivo. Ele tem cinco seções principais:

films — Lista de filmes prontos

É um array de objetos. Cada objeto é um filme com as seguintes propriedades:

```
{
  "title": "O Amor Dá Voltas",
  "titleEn": "O Amor Dá Voltas",
  "director": "Marcos Bernstein",
  "year": "2022",
  "genre": "Longa-Metragem de ficção",
  "hero": false,
  "synopsis": "Os encontros e desencontros amorosos de André...",
  "synopsisEn": "The romantic encounters...",
  "tags": [],
  "imgPortrait": "https://...caminho-da-imagem-retrato...",
  "imgLandscape": "https://...caminho-da-imagem-paisagem...",
  "videoHover": "https://...caminho-do-video-preview...",
  "videoTrailer": "https://youtube.com/...",
  "videoTrailers": ["https://youtube.com/..."],
  "fichatecnica": [
    { "role": "Produção", "name": "Pontos de Fuga" }
  ],
  "elenco": [
    { "role": "Marli", "name": "Dira Paes" }
  ],
  "fotografias": ["https://...foto1...", "https://...foto2..."],
  "makingOff": ["https://...mo1...", "https://...mo2..."]
}
```

Cada propriedade corresponde a algo visível no site:

- title / titleEn — o título em PT e EN
- hero — se true, este filme aparece no carrossel da página inicial
- imgPortrait — a imagem vertical usada nos cards

- `imgLandscape` — a imagem horizontal usada no hero e na página do filme
- `videoHover` — o vídeo que toca quando o mouse passa sobre o card
- `videoTrailers` — lista de trailers (YouTube ou Vimeo)
- `fichatecnica` — array de objetos `{role, name}` com a equipe técnica
- `elenco` — array de objetos `{role, name}` com os atores

`upcomingFilms` — Filmes em produção

Mesma estrutura dos filmes, com uma propriedade extra:

```
"status": "filming"
```

O status pode ser: `"filming"` (filmando), `"dev"` (desenvolvimento) ou `"post"` (pós-produção).

`historiaData` — Conteúdo da página Nossa História

Contém sub-objetos: `manifesto`, `marcos` (linha do tempo), `team` (equipe), `festivals`, `parceiros`.

`otherProductions` — Outras produções

Filmes menores, com estrutura simplificada.

`pagesData` — Textos editáveis das páginas

Permite ao administrador editar textos específicos de cada página sem mexer no código. Exemplo:

```
"pagesData": {
  "index": {
    "ctaTitlePt": "Tem uma história que precisa ser contada?",
    "ctaTitleEn": "Got a story that needs to be told?"
  }
}
```

Capítulo 4 — O Carregador de Dados: `data.js`

O problema que este arquivo resolve

O site precisa dos dados do `data.json` para funcionar. Mas como buscar esse arquivo? Não é possível ler arquivos diretamente do disco pelo navegador por questões de segurança. A solução é fazer uma requisição HTTP — como se o navegador "pedisse" o arquivo para um servidor.

A função `fetch()` faz exatamente isso. Mas há um detalhe crucial: **`fetch` é assíncrona** (veja Capítulo 2.8). O arquivo não chega instantaneamente — pode levar milissegundos ou segundos.

O código completo

```
let films          = [];
let upcomingFilms = [];
let historiaData   = {};
```

```

let otherProductions = [];
let pagesData        = {};

const dataReady = fetch('scripts/data.json')
  .then(res => {
    if (!res.ok) throw new Error(`Erro ao carregar data.json: ${res.status}`);
    return res.json();
  })
  .then(json => {
    films           = json.films;
    upcomingFilms   = json.upcomingFilms;
    historiaData    = json.historiaData;
    otherProductions = json.otherProductions || [];
    pagesData       = json.pagesData        || {};
  })
  .catch(err => {
    console.error('[pontos de fuga] Falha ao carregar dados:', err);
  });

```

Linha a linha

Linhas 1-5 — Declaração das variáveis globais:

```
let films = [];
```

Cria uma variável chamada `films` com um array vazio. Por que declarar antes? Porque outros scripts precisam referenciar essas variáveis. Se você declarar dentro do `.then()`, elas só existiriam lá dentro.

Linhas 7-9 — A requisição:

```
const dataReady = fetch('scripts/data.json')
```

`fetch()` dispara uma requisição para o arquivo. Ela retorna imediatamente uma Promise — o arquivo ainda não chegou, mas a Promise representa a promessa de que vai chegar. Essa Promise é guardada em `dataReady`.

Por que guardar em `dataReady` ? Porque outros scripts vão usar isso:

```

dataReady.then(() => {
  // só executa aqui quando os dados estiverem prontos
  renderizarPagina();
});

```

Linha 10 — Verificação de erro HTTP:

```
if (!res.ok) throw new Error(`Erro ao carregar data.json: ${res.status}`);
```

Uma requisição bem-sucedida retorna status 200. `res.ok` é `true` se o status for entre 200 e 299. Se der 404 (não encontrado) ou 500 (erro do servidor), `res.ok` é `false` e o `throw` dispara um erro, que cai no

```
.catch() .
```

Linha 11 — Converter para JavaScript:

```
return res.json();
```

A resposta chega como texto puro. `res.json()` converte esse texto (que está no formato JSON) para um objeto JavaScript. Isso também é assíncrono — por isso o `return` passa essa nova Promise para o próximo `.then()`.

Linhas 13-18 — Preencher as variáveis:

```
films = json.films;  
otherProductions = json.otherProductions || [];
```

Atribui cada seção do JSON à variável correspondente. Note o `|| []`: se `json.otherProductions` não existir (campo ausente no JSON), usa um array vazio em vez de `undefined`. Isso evita erros quando o código tenta percorrer a lista.

Capítulo 5 — O Código Compartilhado: `script-shared.js`

Por que existe um arquivo compartilhado?

Todas as páginas do site têm os mesmos elementos: cabeçalho (header), menu, rodapé (footer), botão de idioma, botão de tema. Em vez de repetir o mesmo código em 6 arquivos, tudo isso vive em um único arquivo carregado em todas as páginas.

Seção 1 — A Paleta de Cores Cíclica

```
const PALETTE = ['#c9a84c', '#c4622d', '#6b8f71', '#8b1a1a'];  
let headerColorIdx = -1;  
  
menuBtn.addEventListener('mouseenter', () => {  
  headerColorIdx = (headerColorIdx + 1) % PALETTE.length;  
  menuBtn.style.color = PALETTE[headerColorIdx];  
});
```

O que são as cores: dourado (`#c9a84c`), laranja queimado (`#c4622d`), verde musgo (`#6b8f71`), vinho (`#8b1a1a`).

O índice começa em -1. Por quê? Para que a primeira cor usada seja a de índice 0. Se começasse em 0, a primeira cor usada seria a de índice 1.

O operador `%` (módulo): divide um número e retorna o resto. É o mecanismo do ciclo:

```
0 % 4 = 0 → cor 0 (dourado)  
1 % 4 = 1 → cor 1 (laranja)
```

```
2 % 4 = 2 → cor 2 (verde)
3 % 4 = 3 → cor 3 (vinho)
4 % 4 = 0 → cor 0 (dourado) – volta ao início!
5 % 4 = 1 → cor 1 (laranja)
```

O resultado nunca sai de 0 a 3, independente de quantas vezes o mouse passar.

Seção 2 — Internacionalização (i18n)

i18n é uma abreviação de "internationalization" (18 letras entre o i e o n). É o suporte a múltiplos idiomas.

```
const COMMON_I18N = {
  pt: {
    'menu productions': 'Produções',
    'menu upcoming': 'Vem aí',
    'footer copy': '© 2025 - PONTOS DE FUGA',
  },
  en: {
    'menu productions': 'Productions',
    'menu upcoming': 'Coming Soon',
    'footer copy': '© 2025 - PONTOS DE FUGA',
  }
};
```

Este objeto é um dicionário bilíngue. Para cada chave (ex: `'menu productions'`), existe a tradução em PT e em EN. Esse dicionário é **compartilhado** — todas as páginas o usam como base e acrescentam suas próprias chaves.

Como o i18next funciona: é uma biblioteca (um conjunto de funções prontas) que gerencia as traduções. Você registra os dicionários e pede a tradução por chave:

```
i18next.t('menu productions')
// retorna 'Produções' se o idioma for PT
// retorna 'Productions' se o idioma for EN
```

Como os elementos HTML são traduzidos:

No HTML, elementos com `data-i18n` são marcados para tradução:

```
<span data-i18n="menu productions">Produções</span>
```

A função `applyI18n()` percorre todos esses elementos e substitui o texto:

```
function applyI18n() {
  document.querySelectorAll('[data-i18n]').forEach(el => {
    el.textContent = i18next.t(el.getAttribute('data-i18n'));
  });
}
```

Fluxo de troca de idioma:

1. Usuário clica no botão "EN"
2. `i18next.changeLanguage('en')` é chamado
3. O callback chama `window.updateDOM()` (definida em cada script de página)
4. `updateDOM()` chama `applyI18n()` e re-renderiza o conteúdo dinâmico
5. Todos os textos aparecem em inglês

Seção 3 — Scroll Reveal (animação ao rolar)

```
function observeReveal(threshold = 0.1) {
  const io = new IntersectionObserver((entries) => {
    entries.forEach(e => {
      if (e.isIntersecting) {
        e.target.classList.add('visible');
        io.unobserve(e.target);
      }
    });
  }, { threshold });

  document.querySelectorAll('.reveal:not(.visible)').forEach(el => io.observe(el));
}
```

IntersectionObserver é uma API nativa do navegador que monitora quando elementos entram ou saem da área visível da tela (viewport).

Por que não usar o evento `scroll` ? O evento `scroll` dispara dezenas de vezes por segundo enquanto o usuário rola a página — para cada disparo, o navegador teria que calcular a posição de todos os elementos. É muito custoso. O `IntersectionObserver` só avisa quando há uma mudança real.

Como funciona passo a passo:

1. Qualquer elemento com classe `.reveal` no HTML começa invisível (o CSS define `opacity: 0`)
2. `observeReveal()` registra o observador para cada um
3. Quando o elemento fica pelo menos 10% visível (`threshold: 0.1`), a classe `.visible` é adicionada
4. O CSS define uma animação para a classe `.visible` (ex: `opacity: 1` com transição)
5. `io.unobserve(e.target)` — para de monitorar o elemento depois que animou, economizando recursos

Parâmetro com valor padrão:

```
function observeReveal(threshold = 0.1)
```

Se você chamar `observeReveal()` sem argumento, usa 0.1. Se chamar `observeReveal(0.3)`, usa 0.3. O `= 0.1` é o valor padrão.

Seção 4 — Menu Toggle

```
function openMenu() {
  menuOpen = true;
  menuOverlay.classList.add('open');
```

```

    menuBtn.classList.add('open');
    document.body.style.overflow = 'hidden';
}

function closeMenu() {
    menuOpen = false;
    menuOverlay.classList.remove('open');
    menuBtn.classList.remove('open');
    document.body.style.overflow = '';
}

menuBtn.addEventListener('click', () => menuOpen ? closeMenu() : openMenu());

```

A variável `menuOpen` guarda o estado atual: menu aberto ou fechado. Quando o botão é clicado, o código verifica o estado e executa a função oposta.

`document.body.style.overflow = 'hidden'` trava o scroll da página enquanto o menu está aberto — o usuário não rola a página por baixo do menu. `''` (string vazia) restaura o comportamento padrão.

Fechar clicando fora:

```

menuOverlay.addEventListener('click', e => {
    if (e.target === menuOverlay) closeMenu();
});

```

`e.target` é o elemento exato onde o clique aconteceu. Se clicar no item do menu (um `<a>`), `e.target` é o link — não fecha. Se clicar no fundo escuro, `e.target` é o `menuOverlay` — fecha.

Seção 5 — Esconder o Header ao Rolar

```

var lastScrollTop = 0;
var header = document.querySelector('header');

window.addEventListener('scroll', function() {
    var scrollTop = window.pageYOffset || document.documentElement.scrollTop;

    if (scrollTop > lastScrollTop && scrollTop > 50) {
        header.classList.add('header--hidden');
    } else {
        header.classList.remove('header--hidden');
    }
    lastScrollTop = scrollTop;
});

```

A lógica compara a posição atual com a última posição gravada:

- `scrollTop > lastScrollTop` → está rolando para baixo
- `scrollTop > 50` → já passou dos primeiros 50px (ignora micro-movimentos no topo)
- Rolando para baixo além de 50px → esconde o header
- Rolando para cima → mostra o header

Seção 6 — O Contrato `window.updateDOM`

```
// No script-shared.js, o botão de idioma chama:
if (typeof window.updateDOM === 'function') window.updateDOM();

// No script de cada página, a função é definida:
window.updateDOM = function() {
  applyI18n();
  renderizarPagina();
}
```

Este é um padrão de design chamado **contrato de interface**. O `script-shared.js` promete: "quando o idioma mudar, vou chamar `window.updateDOM()` ." Cada script de página promete: "vou definir `window.updateDOM()` com o que precisa ser atualizado."

`window` é o objeto global do navegador — qualquer propriedade colocada nele é acessível de qualquer lugar. `typeof window.updateDOM === 'function'` verifica se a função foi definida antes de chamá-la, evitando erro.

Capítulo 6 — A Página Inicial: `script-index.js`

Visão geral

Este é o script mais rico em funcionalidades. Ele controla:

1. **Stripe** — a faixa de texto animado
2. **Hero Slideshow** — o carrossel de filmes em tela cheia
3. **Preview Grid** — a grade com os 4 primeiros filmes
4. **Upcoming** — a lista de filmes em produção

Todo o código fica dentro de `dataReady.then(() => { ... })` — só começa a rodar depois que os dados do JSON chegaram.

A estrutura de inicialização

```
dataReady.then(() => {

  // ... definição de todas as funções ...

  window.updateDOM = function() {
    applyI18n();
    renderIdentity();
    renderPreview();
    renderUpcoming();
    goToSlide(currentSlide);
  }

  i18next.init({
```

```

    lng: 'pt',
    resources: {
      pt: { translation: { ...COMMON_I18N.pt, 'hero.label': 'Em destaque · 2025',
... } },
      en: { translation: { ...COMMON_I18N.en, 'hero.label': 'Featured · 2025',
... } }
    }
  }, () => {
    // callback: executado quando o i18next estiver pronto
    applyI18n();
    goToSlide(0);
    renderPreview();
    renderUpcoming();
    observeReveal(0.15);
  });
});

```

A ordem importa:

1. `dataReady` garante que os dados existem
2. Dentro dele, as funções são definidas mas não executadas ainda
3. `i18next.init()` prepara as traduções
4. No callback do `i18next.init()`, as funções são chamadas pela primeira vez

O Hero Slideshow

Gerando os slides dinamicamente

```
const heroFilms = films.filter(f => f.hero);
```

`filter()` cria uma nova lista só com os filmes que têm `hero: true` no JSON. Se 3 filmes tiverem `hero: true`, `heroFilms` terá 3 elementos.

```
slidesContainer.innerHTML = heroFilms.map((_, i) =>
  `<div class="hero-slide${i === 0 ? ' active' : ''}></div>`
).join('');
```

`map()` transforma cada filme numa string HTML. O `_` significa "não preciso do filme em si, só do índice `i`". O primeiro slide recebe a classe `active`, os outros não. `.join('')` junta as strings sem separador.

O resultado no HTML seria algo como:

```
<div class="hero-slide active"></div>
<div class="hero-slide"></div>
<div class="hero-slide"></div>
```

A função `goToSlide`


```

let currentSlide = 0;

function goToSlide(n) {
  // Remove 'active' do slide atual
  slides[currentSlide].classList.remove('active');
  dots[currentSlide].classList.remove('active');

  // Atualiza o estado
  currentSlide = n;

  // Adiciona 'active' ao novo slide
  slides[currentSlide].classList.add('active');
  dots[currentSlide].classList.add('active');

  // Atualiza imagem e texto
  const f = heroFilms[n];
  if (f.imgLandscape) {
    slides[n].style.backgroundImage = `url('${f.imgLandscape}')`;
  }
  document.querySelector('.hero-title').innerHTML = f.title;
  document.querySelector('.hero-meta').textContent =
    `Dir. ${f.director} · ${f.genre || 'Drama'} · ${f.year}`;
}

```

`currentSlide` é a **variável de estado** — ela lembra qual slide está ativo. A função usa essa variável para saber de onde sair e usa o parâmetro `n` para saber para onde ir.

Autoplay e interação manual

```

// Clique nos dots
dots.forEach(d => d.addEventListener('click', () => goToSlide(+d.dataset.index)));

// Autoplay a cada 5 segundos
setInterval(() => goToSlide((currentSlide + 1) % heroFilms.length), 5000);

```

`+d.dataset.index` — o `+` converte uma string para número. `d.dataset.index` lê o atributo `data-index` do HTML como texto; o `+` garante que seja um número antes de passar para `goToSlide`.

`setInterval()` executa uma função repetidamente num intervalo. Aqui: a cada 5000ms (5 segundos), avança para o próximo slide. O `%` garante o ciclo infinito.

O Preview Grid

```

function renderPreview() {
  const previewFilms = films.slice(0, 4);

  document.getElementById('previewGrid').innerHTML = previewFilms.map(f => {
    const originalIndex = films.indexOf(f);
    return `

```

```

    <a href="filme.html?i=${originalIndex}" class="preview-card reveal">
      
      <div class="preview-card__info">
        <div class="preview-card__year">${f.year}</div>
        <div class="preview-card__title">${lang === 'en' ? f.titleEn : f.title}
</div>
      </div>
    </a>`;
  })).join('');

  observeReveal(0.15);
}

```

`films.slice(0, 4)` pega os primeiros 4 filmes do array sem modificar o original.

`films.indexOf(f)` retorna a posição do filme no array original. Isso é necessário porque o link `filme.html?i=2` precisa do índice **original** para a página do filme encontrar o dado correto. Se usássemos o índice do `slice`, o link quebraria quando a ordem dos filmes mudasse.

`onerror="this.style.display='none'"` — se a imagem não carregar (URL quebrada), ela simplesmente `some`. `this` dentro de um atributo de evento HTML refere-se ao próprio elemento.

A Lista Upcoming

```

function renderUpcoming() {
  document.getElementById('upcomingList').innerHTML = upcomingFilms.map((f, i) => {
    const status = i18next.t(statusKey[f.status]);
    const num    = String(i + 1).padStart(2, '0');
    const delay  = i > 0 ? `reveal-delay-${i}` : '';

    return `
      <li class="upcoming-item reveal ${delay}">
        <a href="filme.html?src=upcoming&i=${i}">
          <span class="upcoming-item__num">${num}</span>
          <span class="upcoming-item__title">${f.title}</span>
          <div class="upcoming-item__status">${status}</div>
        </a>
      </li>`;
  })).join('');
}

```

`statusKey[f.status]` — acessa o mapa de status definido no `shared`:

```

const statusKey = { filming: 'status.filming', dev: 'status.dev', post:
'status.post' };

```

Se `f.status` é `'filming'`, `statusKey['filming']` retorna `'status.filming'`, e `i18next.t('status.filming')` retorna `'Filmando'` ou `'Filming'`.

`padStart(2, '0')` — formata números como `01`, `02`, `10`. O número 1 vira `'01'`.

`reveal-delay- $\{i\}$` — classes CSS que adicionam atraso progressivo na animação. O segundo item aparece ligeiramente depois do primeiro, criando um efeito cascata.

Capítulo 7 — A Página de Produções: `script-producoes.js`

Visão geral

Esta página mostra todos os filmes em um grid, com opção de ordenação. Os filmes têm um preview em vídeo ao passar o mouse.

Ordenação do Grid

```
function renderGrid(sortMode = 'relevance') {
  let sorted = [...films]; // cria uma cópia

  if (sortMode === 'recent') {
    sorted.sort((a, b) => b.year - a.year); // mais recente primeiro
  } else if (sortMode === 'az') {
    sorted.sort((a, b) => a.title.localeCompare(b.title, 'pt', { sensitivity: 'base' }));
  }
  // 'relevance' não ordena — mantém a ordem original do JSON

  document.querySelector('.films-grid').innerHTML = sorted.map((film, i) => {
    const originalIndex = films.indexOf(film);
    return createCard(film, originalIndex);
  }).join('');

  attachCardHovers();
}
```

Por que `[...films]` antes de ordenar?

A função `.sort()` modifica o array original (em programação dizemos que ela "muta" o array). Se fizéssemos `films.sort(...)`, a ordem original seria perdida para sempre. Criando uma cópia com `[...films]`, o array `films` permanece intacto para a próxima vez que a função for chamada com um modo diferente.

Como o `.sort()` funciona:

`.sort()` recebe uma função de comparação com dois parâmetros (`a` e `b`):

- Retorna número negativo → `a` vem antes de `b`
- Retorna número positivo → `b` vem antes de `a`
- Retorna zero → mesma posição

```
// Ordem decrescente por ano (mais recente primeiro)
sorted.sort((a, b) => b.year - a.year);
// Se a=2020 e b=2024: 2024 - 2020 = 4 (positivo) → b vem antes → 2024, 2020
```

localeCompare — comparação de texto respeitando o idioma:

```
a.title.localeCompare(b.title, 'pt', { sensitivity: 'base' })
```

Um simples `a > b` não funciona bem para texto com acentos e caracteres especiais em português.

`localeCompare` sabe que `ç` vem depois de `c` e que acentos não mudam a ordem de classificação (sensitivity: 'base').

O Card de Filme

```
function createCard(film, index) {
  const wide      = film.size === 'wide' ? 'card--wide' : '';
  const hasVideo  = !!film.videoHover;

  return `
    <a href="filme.html?i=${index}" class="film-card ${wide}">
      <div class="film-card__img">
        ${placeholderSVG()}
        
        ${hasVideo ? `<video src="${film.videoHover}" muted playsinline
preload="none"></video>` : ''}
      </div>
      <div class="film-card__info">
        <div class="film-card__title">${film.title}</div>
        <div class="film-card__dir">Dir. ${film.director}</div>
        <span class="film-card__year">${film.year}</span>
      </div>
    </a>`;
}
```

!!film.videoHover — o duplo `!` converte qualquer valor para booleano (`true` ou `false`). Uma string não vazia é `true`. Uma string vazia `''` ou `undefined` é `false`. Se o filme tem vídeo, inclui o elemento `<video>` no HTML; caso contrário, não inclui nada.

Layout do primeiro card:

```
const pattern = i === 0 ? { size: 'wide', ratio: 'l', rows: 1 }
                        : { size: '',    ratio: 'p', rows: 1 };
return createCard({ ...film, ...pattern }, originalIndex);
```

`{ ...film, ...pattern }` — mescla os dados do filme com o padrão de layout. O primeiro filme recebe `size: 'wide'` (ocupa duas colunas), os demais ficam no tamanho padrão. O `...pattern` sobrescreve as propriedades `size`, `ratio` e `rows` do objeto original.

Hover com Vídeo e Re-attach de Eventos

```
function attachCardHovers() {
  document.querySelectorAll('.film-card').forEach(card => {
    card.addEventListener('mouseenter', () => {
      const video = card.querySelector('.film-card__video');
      if (video) { video.currentTime = 0; video.play(); }
    });
    card.addEventListener('mouseleave', () => {
      const video = card.querySelector('.film-card__video');
      if (video) { video.pause(); video.currentTime = 0; }
    });
  });
}
```

Por que `attachCardHovers()` é chamada depois de cada `renderGrid()` ?

Quando o `innerHTML` de um container é substituído, todos os elementos dentro dele são destruídos e recriados. Os novos elementos **não têm** os event listeners dos antigos. A função `attachCardHovers()` precisa ser chamada toda vez que o grid é renderizado para registrar os eventos nos novos elementos.

`video.currentTime = 0` no `mouseenter` — faz o vídeo sempre começar do início, mesmo que o usuário tenha passado o mouse por cima e saído antes do vídeo terminar.

Capítulo 8 — A Página de Cada Filme: `script-filme.js`

Visão geral

Uma única página HTML (`filme.html`) serve para **todos** os filmes. O JavaScript lê qual filme mostrar pela URL.

Lendo Parâmetros da URL

```
function getFilme() {
  const params = new URLSearchParams(window.location.search);
  const idx    = params.get('i');
  const src    = params.get('src');

  const source = src === 'upcoming' ? upcomingFilms
    : src === 'other'      ? otherProductions
    :                        films;

  return source[parseInt(idx)] || null;
}
```

Quando você clica num filme, o link é algo como: `filme.html?i=2&src=upcoming`

`window.location.search` retorna a parte da URL depois do `?`: `"?i=2&src=upcoming"`

`URLSearchParams` é uma API nativa que parseia essa string:

```
params.get('i')    // retorna "2" (string)
params.get('src')  // retorna "upcoming"
```

`parseInt("2")` converte a string `"2"` para o número `2`, necessário para indexar o array.

O operador ternário encadeado decide de qual array buscar:

- `src === 'upcoming'` → busca em `upcomingFilms`
- `src === 'other'` → busca em `otherProductions`
- Nenhum dos dois → busca em `films`

`|| null` — se o índice não existir no array, retorna `null` em vez de `undefined`.

Convertendo URLs de Vídeo para Embed

O YouTube e o Vimeo não permitem que você incorpore um vídeo com a URL normal de assistir. Eles têm URLs especiais de "embed". Este código converte automaticamente:

```
function toEmbedUrl(url) {
  // Detecta YouTube: youtube.com/watch?v=XXXXXXX ouyoutu.be/XXXXXXX
  const ytMatch = url.match(/(?:youtube\.com\/watch\?.*v=|youtu\.be\/)([A-Za-z0-9_-]{11})/);
  if (ytMatch) return `https://www.youtube.com/embed/${ytMatch[1]}`;

  // Detecta Vimeo: vimeo.com/123456789
  const vmMatch = url.match(/vimeo\.com\/(\d+)/);
  if (vmMatch) return `https://player.vimeo.com/video/${vmMatch[1]}`;

  return null; // não é YouTube nem Vimeo — usa como está
}
```

Regex (expressão regular): é uma linguagem para descrever padrões em texto. Parece intimidante, mas a ideia é simples: encontrar o ID do vídeo dentro da URL.

Para o YouTube, o ID tem sempre 11 caracteres (letras, números, `-` ou `_`). O trecho `[A-Za-z0-9_-]{11}` significa: "11 caracteres que sejam letra, número, underline ou hífen".

O `.match()` retorna um array. O índice `[1]` contém o que estava entre parênteses na regex (o grupo de captura) — que é exatamente o ID do vídeo.

Suporte a Formatos Antigos e Novos

```
const allTrailers = (f.videoTrailers && f.videoTrailers.length)
  ? f.videoTrailers.filter(Boolean)
  : f.videoTrailer ? [f.videoTrailer] : [];
```

O sistema evoluiu. No início, cada filme tinha um único trailer (`videoTrailer` : string). Depois, passou a suportar múltiplos trailers (`videoTrailers` : array).

Este código suporta os dois formatos:

1. Se `videoTrailers` existir e tiver itens → usa esse array
2. Senão, se `videoTrailer` existir → coloca numa lista de um elemento: `[f.videoTrailer]`
3. Senão → lista vazia

`.filter(Boolean)` remove itens "vazios" da lista (strings vazias, `null`, `undefined`).

Seções Condicionais

```
function renderCrewSection(wrapId, bodyId, items) {
  const wrap = document.getElementById(wrapId);
  const body = document.getElementById(bodyId);
  if (items && items.length) {
    body.innerHTML = items.map(item => `
      <div class="filme-crew-row">
        <span>${item.role} || ''</span>
        <span>${item.name} || ''</span>
      </div>` ).join('');
    wrap.style.display = ''; // mostra
  } else {
    wrap.style.display = 'none'; // esconde
  }
}

renderCrewSection('filmeFichaTecnica', 'filmeFichaTecnicaBody', f.fichatecnica);
renderCrewSection('filmeElenco', 'filmeElencoBody', f.elenco);
```

Ficha técnica e elenco têm a mesma lógica: "se existir e tiver itens, renderiza e mostra; caso contrário, esconde." Em vez de duplicar esse código, criou-se uma função que recebe os IDs e os dados como parâmetros, e é chamada duas vezes.

`wrap.style.display = ''` — atribuir string vazia ao display restaura o valor padrão definido no CSS (normalmente `block` ou `flex`).

O Lightbox de Galeria

O lightbox é a janela que abre quando você clica numa foto. Ele mostra a imagem em tamanho maior e permite navegar entre as fotos.

Estado do Lightbox

```
let lbImages = []; // todas as imagens da galeria atual
let lbIndex = 0; // qual imagem está sendo mostrada
```

O lightbox tem seu próprio estado, separado do resto da página.

Abrindo o Lightbox

```
function lbOpen(images, startIdx) {
  lbImages = images;
  lbIndex = startIdx;
  lbShow();
  lightbox.classList.add('open');
  document.body.style.overflow = 'hidden';
}
```

Recebe todas as imagens da galeria e o índice da que foi clicada. Chama `lbShow()` para exibir a imagem, torna o lightbox visível e trava o scroll.

Mostrando a Imagem Atual

```
function lbShow() {
  lbImg.src = lbImages[lbIndex];
  lbCounter.textContent = `${lbIndex + 1} / ${lbImages.length}`;
  lbPrev.disabled = lbIndex === 0;
  lbNext.disabled = lbIndex === lbImages.length - 1;
}
```

`lbIndex + 1` — como arrays começam em 0, soma 1 para exibir ao usuário (exibir "1 / 5" em vez de "0 / 4").

`lbPrev.disabled = lbIndex === 0` — desabilita o botão "anterior" na primeira foto. `disabled` é uma propriedade booleana dos botões HTML: `true` desabilita, `false` habilita.

Navegação por Teclado

```
document.addEventListener('keydown', e => {
  if (!lightbox.classList.contains('open')) return;
  if (e.key === 'Escape')    lbClose_();
  if (e.key === 'ArrowLeft' && lbIndex > 0) { lbIndex--;
  lbShow(); }
  if (e.key === 'ArrowRight' && lbIndex < lbImages.length - 1) { lbIndex++;
  lbShow(); }
});
```

O listener fica ativo em toda a página, mas a primeira linha verifica se o lightbox está aberto. Se não estiver, `return` para a execução imediatamente (não faz nada).

`&&` é o operador lógico "E". `e.key === 'ArrowLeft' && lbIndex > 0` significa: "se a tecla for seta esquerda E não for a primeira imagem". A segunda condição evita que o índice vá abaixo de 0.

Capítulo 9 — Nossa História: `script-historia.js`

Visão geral

Esta é a página mais "estática" — ela renderiza dados estruturados sem grandes interações. É um bom exemplo para entender renderização de dados complexos.

Seções da Página

A função `renderHistoria()` preenche quatro seções: manifesto, festivais, marcos (linha do tempo) e time.

Manifesto — Escrevendo em dois elementos de uma vez

```
const [p1, p2] = document.querySelectorAll('.historia-manifesto__body p');
p1.textContent = lang === 'en' ? d.manifesto.p1En : d.manifesto.p1;
p2.textContent = lang === 'en' ? d.manifesto.p2En : d.manifesto.p2;
```

`document.querySelectorAll('.historia-manifesto__body p')` retorna uma lista de todos os `<p>` dentro de `.historia-manifesto__body`.

`const [p1, p2] = ...` é **array destructuring** — pega o primeiro elemento e chama de `p1`, o segundo de `p2`. É equivalente a:

```
const elementos = document.querySelectorAll('...');
const p1 = elementos[0];
const p2 = elementos[1];
```

Festivais — Completando o Grid

O grid de festivais usa CSS com 6 colunas. Se tivermos 7 festivais, a segunda linha ficaria com apenas 1 elemento — visualmente desequilibrado. O código calcula quantos elementos vazios ("fillers") adicionar:

```
const rem = units % 6;
const festivaisFillerCount = rem === 0 ? 0 : 6 - rem;
```

Se tivermos 7 itens: $7 \% 6 = 1$. Precisamos de $6 - 1 = 5$ fillers para completar 6. Se tivermos 12 itens: $12 \% 6 = 0$. Não precisamos de nenhum filler.

```
'<div class="historia-festival--filler"></div>'.repeat(festivaisFillerCount)
```

`.repeat(n)` repete uma string `n` vezes.

Time — Layout Alternado

```
d.team.map((m, i) => `
  <div class="historia-member${i % 2 !== 0 ? ' historia-member--reverse' : ''}>
```

`i % 2 !== 0` — se o índice for ímpar (1, 3, 5...), adiciona a classe `--reverse`. O CSS espelha o layout para essa classe. Resultado: membros em posições pares têm foto à esquerda, em posições ímpares à direita — efeito zigue-zague automático.

Parceiros — String ou Objeto

```
parceiros.map(p => {
  const name = typeof p === 'string' ? p : (p.name || '');
  const logo = typeof p === 'object' && p.logo ? p.logo : '';
  ...
});
```

O array de parceiros pode ter dois formatos: uma string simples ("Nome do Parceiro") ou um objeto com nome e logo ({ name: "...", logo: "..." }). O código trata os dois casos com `typeof`.

Capítulo 10 — Vem Aí: `script-vemai.js`

Visão geral

Esta é a página mais simples. Lista os filmes em produção com status e sinopse.

Estrutura do Card

```
upcomingFilms.map((f, i) => {
  const status = i18next.t(statusKey[f.status]);

  return `
    <a href="filme.html?src=upcoming&i=${i}" class="vemai-filme reveal">
      <div class="vemai-filme__status">${status}</div>
      <h2 class="vemai-filme__title">${f.title}</h2>
      <div class="vemai-filme__meta">${f.genre} · ${f.year}</div>
      <p class="vemai-filme__synopsis">${f.synopsis}</p>
    </a>`;
})
```

`href="filme.html?src=upcoming&i=${i}"` — o link passa dois parâmetros:

- `src=upcoming` — indica que o `script-filme.js` deve buscar em `upcomingFilms`
- `i=${i}` — o índice do filme nesse array

Se o usuário clicar no segundo item (`i=1`), a URL gerada seria `filme.html?src=upcoming&i=1`. O `script-filme.js` lê isso e busca `upcomingFilms[1]`.

Capítulo 11 — Contato: `script-contato.js`

Visão geral

Esta página tem um formulário que envia e-mails usando o serviço **EmailJS** — uma solução que permite enviar e-mails direto do navegador, sem servidor.

O Fluxo do Formulário

```

form.addEventListener('submit', e => {
  e.preventDefault();

  // 1. Lê os campos
  const nome      = form.nome.value.trim();
  const email     = form.email.value.trim();
  const assunto   = form.assunto.value.trim();
  const mensagem  = form.mensagem.value.trim();

  // 2. Valida
  if (!nome || !email || !assunto || !mensagem) {
    feedback.textContent = 'Preencha todos os campos obrigatórios.';
    feedback.className = 'form-feedback error';
    return;
  }

  // 3. Desabilita o botão
  const btn = form.querySelector('.btn-submit');
  btn.disabled = true;
  btn.style.opacity = '0.5';

  // 4. Envia
  emailjs.send('service_id', 'template_id', { nome, email, assunto, mensagem })
    .then(() => {
      feedback.textContent = 'Mensagem enviada.';
      feedback.className = 'form-feedback success';
      form.reset();
    })
    .catch(() => {
      feedback.textContent = 'Algo deu errado.';
      feedback.className = 'form-feedback error';
    })
    .finally(() => {
      btn.disabled = false;
      btn.style.opacity = '1';
    });
});

```

e.preventDefault()

Por padrão, um formulário HTML navega para outra página ao ser submetido. `e.preventDefault()` cancela esse comportamento — o JavaScript assume o controle.

.trim()

Remove espaços em branco do início e fim de uma string. `" Pedro ".trim() → "Pedro"`. Evita que campos preenchidos só com espaços passem na validação.

Validação

```
if (!nome || !email || !assunto || !mensagem) { ... }
```

Em JavaScript, uma string vazia ("") é considerada `false`. O `!` inverte: `!""` é `true`. Então a condição é: "se nome estiver vazio OU email estiver vazio OU...". Se qualquer campo estiver vazio, mostra o erro e `return` para a execução — não envia.

Estado do Botão Durante o Envio

```
btn.disabled = true;    // desabilita (evita duplo clique)
btn.style.opacity = '0.5'; // feedback visual

emailjs.send(...)
  .then(...)
  .catch(...)
  .finally(() => {
    btn.disabled = false;    // reabilita sempre, independente de sucesso ou erro
    btn.style.opacity = '1';
  });
```

`.finally()` é executado **sempre**, seja sucesso ou erro. É o lugar certo para "limpar" o estado — aqui, reabilitar o botão.

Feedback Visual por Classe

```
feedback.className = 'form-feedback error';    // fundo vermelho (CSS define)
feedback.className = 'form-feedback success';  // fundo verde (CSS define)
```

Em vez de manipular cores diretamente no JavaScript, apenas troca a classe. O CSS define como cada estado parece. Esta separação é uma boa prática: o JavaScript decide **o que** aconteceu; o CSS decide **como** parece.

Capítulo 12 — Glossário de Termos Técnicos

Termo	Definição
API	Interface de Programação de Aplicação. Um conjunto de regras para dois sistemas se comunicarem. A API do GitHub permite que o painel admin salve dados no repositório.
Array	Uma lista ordenada de itens em programação.
Assíncrono	Código que não espera uma operação terminar para continuar. O resultado chega depois, via callback ou Promise.
Boolean	Um tipo de dado que só tem dois valores: <code>true</code> (verdadeiro) ou <code>false</code> (falso).
Callback	Uma função passada como argumento para outra função, para ser chamada quando uma operação terminar.

CSS	Cascading Style Sheets. Linguagem para definir a aparência de elementos HTML.
data-i18n	Atributo HTML personalizado que marca um elemento para ser traduzido pelo i18next.
DOM	Document Object Model. A representação em memória da estrutura de uma página HTML, que o JavaScript pode ler e modificar.
Event Listener	Uma função registrada para executar quando um evento específico acontecer (clique, scroll, etc).
Fallback	Um valor ou comportamento alternativo usado quando o preferido não está disponível.
fetch()	Função nativa do JavaScript para fazer requisições HTTP e buscar recursos da rede.
Filler	Elemento vazio inserido para completar uma grade ou layout.
Front-end	A parte do sistema que roda no navegador do usuário.
HTML	HyperText Markup Language. Linguagem para estruturar o conteúdo de páginas web.
i18n	Abreviação de "internationalization". Suporte a múltiplos idiomas em um sistema.
i18next	Biblioteca JavaScript de internacionalização usada no projeto.
IntersectionObserver	API do navegador para detectar quando um elemento entra na área visível da tela.
JavaScript	Linguagem de programação executada no navegador. Responsável pelo comportamento dinâmico das páginas.
JSON	JavaScript Object Notation. Formato de texto para armazenar dados estruturados.
Lightbox	Componente de interface que exibe imagens em tamanho ampliado sobre a página.
map()	Método de array que transforma cada item em outra coisa, retornando um novo array.
Módulo (%)	Operador matemático que retorna o resto de uma divisão. Usado para criar ciclos.
Mutação	Modificação de um valor existente no lugar, em vez de criar um novo.
NodeList	O tipo de objeto retornado por <code>querySelectorAll</code> . Similar a um array, mas não idêntico.
Objeto	Uma coleção de propriedades nomeadas (<code>chave: valor</code>).
**`	

&& (AND)	Operador lógico "E". Ambas as condições precisam ser verdadeiras.
Parâmetro	Um valor que uma função recebe para trabalhar. Definido na declaração da função.
Promise	Objeto que representa um valor que ainda não existe mas vai existir. Resultado de operações assíncronas.
Query String	A parte da URL depois do ?. Ex: filme.html?i=2&src=upcoming.
Regex	Regular Expression. Linguagem para descrever padrões em texto e fazer buscas complexas.
Renderizar	Gerar o HTML e inserir na página. "Renderizar um card" significa criar o HTML do card e colocá-lo na tela.
return	Encerra uma função e devolve um valor para quem a chamou.
Spread (...)	Operador que "espalha" os itens de um array ou propriedades de um objeto.
trim()	Método de string que remove espaços em branco do início e do fim.
typeof	Operador que retorna o tipo de uma variável como string: "string", "number", "function", etc.
URL	Endereço de um recurso na internet.
URLSearchParams	API nativa para ler e manipular os parâmetros da query string de uma URL.
Variável	Um nome que representa um valor armazenado em memória.
Viewport	A área visível da página no navegador.
window	O objeto global do navegador. Propriedades em window são acessíveis de qualquer lugar.